



方法二

利用 $a * b \bmod p = a * b - [a * b / p] * p$, 其中 $[]$ 表示下取整。

首先, 当 $a, b < p$ 时, $a * b / p$ 下取整以后一定也小于 p 。我们可以用浮点数执行 $a * b / p$ 的运算, 而不用关心小数点之后的部分。浮点类型 `long double` 在十进制下的有效数字有 18~19 位, 足够胜任。当浮点数的精度不足以保存精确数值时, 它会像科学计数法一样舍弃低位, 正好符合我们的要求。

另外, 虽然 $a * b$ 和 $[a * b / p] * p$ 可能很大, 但是二者的差一定在 $0 \sim p - 1$ 之间, 我们只关心它们较低的若干位即可。所以, 我们可以用 `long long` 来保存 $a * b$ 和 $[a * b / p] * p$ 各自的结果。整数运算溢出相当于舍弃高位, 也正好符合我们的要求。

```
long long mul(long long a, long long b, long long p) {  
    a %= p, b %= p; // 当 a, b 一定在 0~p 之间时, 此行不必要  
    long long c = (long double)a * b / p;  
    long long ans = a * b - c * p;  
    if (ans < 0) ans += p;  
    else if (ans >= p) ans -= p;  
    return ans;  
}
```

◆ 二进制状态压缩

二进制状态压缩, 是指将一个长度为 m 的 `bool` 数组用一个 m 位二进制整数表示并存储的方法。利用下列位运算操作可以实现原 `bool` 数组中对应下标元素的存取。

操作	运算
取出整数 n 在二进制表示下的第 k 位	$(n \gg k) \& 1$
取出整数 n 在二进制表示下的第 $0 \sim k - 1$ 位 (后 k 位)	$n \& ((1 \ll k) - 1)$
把整数 n 在二进制表示下的第 k 位取反	$n \text{ xor } (1 \ll k)$
对整数 n 在二进制表示下的第 k 位赋值 1	$n \mid (1 \ll k)$
对整数 n 在二进制表示下的第 k 位赋值 0	$n \& (\sim(1 \ll k))$

这种方法运算简便, 并且节省了程序运行的时间和空间。当 m 不太大时, 可以直接使用一个整数类型存储。当 m 较大时, 可以使用若干个整数类型 (`int` 数组), 也可以直接利用 C++ STL 为我们提供的 `bitset` 实现 (第 0x71 节)。

【例题】最短 Hamilton 路径 CH0103

给定一张 $n(n \leq 20)$ 个点的带权无向图, 点从 $0 \sim n - 1$ 标号, 求起点 0 到终点





$n-1$ 的最短 Hamilton 路径。

Hamilton 路径的定义是从 0 到 $n-1$ 不重不漏地经过每个点恰好一次。

*扩展题目: POJ2288 Islands and Bridges

很容易想到本题的一种“朴素”^①做法, 就是枚举 n 个点的全排列, 计算路径长度取最小值, 时间复杂度为 $O(n * n!)$, 使用下面的二进制状态压缩 DP 可以优化到 $O(n^2 * 2^n)$ 。关于状态压缩动态规划, 我们将在第 0x56 节详细讲解。

在任意时刻如何表示哪些点已经被经过, 哪些点没有被经过? 可以使用一个 n 位二进制数, 若其第 i 位 ($0 \leq i < n$) 为 1, 则表示第 i 个点已经被经过, 反之未被经过。在任意时刻还需要知道当前所处的位置, 因此我们可以使用 $F[i, j]$ ($0 \leq i < 2^n, 0 \leq j < n$) 表示“点被经过的状态”对应的二进制数为 i , 且目前处于点 j 时的最短路径。

在起点时, 有 $F[1, 0] = 0$, 即只经过了点 0 (i 只有第 0 位为 1), 目前处于起点 0, 最短路径长度为 0。方便起见, 我们将 F 数组其他的值设为无穷大。最终目标是 $F[(1 \ll n) - 1, n - 1]$, 即经过所有点(i 的所有位都是 1), 处于终点 $n - 1$ 的最短路。

在任意时刻, 有公式 $F[i, j] = \min\{F[i \text{ xor } (1 \ll j), k] + \text{weight}(k, j)\}$, 其中 $0 \leq k < n$ 并且 $((i \gg j) \& 1) = 1$, 即当前时刻“被经过的点的状态”对应的二进制数为 i , 处于点 j 。因为 j 只能被恰好经过一次, 所以一定是刚刚经过的, 故在上一时刻“被经过的点的状态”对应的二进制数的第 j 位应该赋值为 0, 也就是 $i \text{ xor } (1 \ll j)$ 。另外, 上一时刻所处的位置可能是 $i \text{ xor } (1 \ll j)$ 中任意一个是 1 的数位 k , 从 k 走到 j 需经过 $\text{weight}(k, j)$ 的路程, 可以考虑所有这样的 k 取最小值。这就是该公式的由来。

```
int f[1 << 20][20];
int hamilton(int n, int weight[20][20]) {
    memset(f, 0x3f, sizeof(f));
    f[1][0] = 0;
    for (int i = 1; i < 1 << n; i++)
        for (int j = 0; j < n; j++) if ((i >> j) & 1)
            for (int k = 0; k < n; k++) if ((i ^ (1 << j)) >> k & 1)
                f[i][j] = min(f[i][j], f[i ^ (1 << j)][k] + weight[k][j]);
    return f[(1 << n) - 1][n - 1];
}
```

提醒: 一些运算符优先级从高到低的顺序如下表所示。最需要注意的地方是: 大小关系比较的符号优先于“位与”“异或”“位或”运算。在程序实现时, 如果不确定优先

^① 朴素算法, 英文术语为 brute-force, 也可直译为“暴力”求解。



级，建议加括号保证运算顺序的正确性。

加减	移位	比较大小	位与	异或	位或
$+, -$	$\ll, \gg$	$>, <, ==, !=$	$\&$	xor (C++ ^)	$ $

◆ 成对变换

通过计算可以发现，对于非负整数 n ：

当 n 为偶数时， $n \text{ xor } 1$ 等于 $n + 1$ 。

当 n 为奇数时， $n \text{ xor } 1$ 等于 $n - 1$ 。

因此，“0 与 1”“2 与 3”“4 与 5”...关于 $\text{xor } 1$ 运算构成“成对变换”。

这一性质经常用于图论邻接表中边集的存储。在具有无向边（双向边）的图中把一对正反方向的边分别存储在邻接表数组的第 n 与 $n + 1$ 位置（其中 n 为偶数），就可以通过 $\text{xor } 1$ 的运算获得与当前边 (x, y) 反向的边 (y, x) 的存储位置。详细应用我们将在讲解邻接表（第 0x13 节）时给出。

◆ lowbit 运算

$\text{lowbit}(n)$ 定义为非负整数 n 在二进制表示下“最低位的 1 及其后边所有的 0”构成的数值。例如 $n = 10$ 的二进制表示为 $(1010)_2$ ，则 $\text{lowbit}(n) = 2 = (10)_2$ 。下面我们来推导 $\text{lowbit}(n)$ 的公式。

设 $n > 0$ ， n 的第 k 位是 1，第 $0 \sim k - 1$ 位都是 0。

为了实现 lowbit 运算，先把 n 取反，此时第 k 位变为 0，第 $0 \sim k - 1$ 位都是 1。再令 $n = n + 1$ ，此时因为进位，第 k 位变为 1，第 $0 \sim k - 1$ 位都是 0。

在上面的取反加 1 操作后， n 的第 $k + 1$ 到最高位恰好与原来相反，所以 $n \& (\sim n + 1)$ 仅有第 k 位为 1，其余位都是 0。而在补码表示下， $\sim n = -1 - n$ ，因此：

$$\text{lowbit}(n) = n \& (\sim n + 1) = n \& (-n)$$

lowbit 运算配合 Hash（第 0x14 节）可以找出整数二进制表示下所有是 1 的位，所花费的时间与 1 的个数同级。为了达到这一目的，我们只需不断把 n 赋值为 $n - \text{lowbit}(n)$ ，直至 $n = 0$ 。例如 $n = 9 = (1001)_2$ ，则 $\text{lowbit}(9) = 1$ 。把 n 赋值为 $9 - \text{lowbit}(9) = 8 = (1000)_2$ ，则 $\text{lowbit}(8) = 8 = (1000)_2$ 。此时 $8 - \text{lowbit}(8) = 0$ ，停止循环。在整个过程中我们减掉了 1 和 $8 = (1000)_2$ 两个数，它们恰好是 n 每一位上的 1 后面补 0 得到的数值。取 1 和 8 的对数 $\log_2 1$ 和 $\log_2 8$ ，即可得知 n 的第 0 位和第 3 位是 1。因为 C++ `math.h` 库的 `log` 函数是以 e 为底的实数运算，并且复杂度





常数较大, 所以我们需要预处理一个数组, 通过 Hash 的方法代替 $\log$ 运算。当 n 较小时, 最简单的方法是直接建立一个数组 H , 令 $H[2^k] = k$, 如下面的程序所示。

```
const MAX_N = 1 << 20;
int H[MAX_N + 1];
for (int i = 0; i <= 20; i++) H[1 << i] = i; // 预处理
while (cin >> n) { // 对多次询问进行求解
    while (n > 0) {
        cout << H[n & -n] << ' ';
        n -= n & -n;
    }
    cout << endl;
}
```

稍微复杂但效率更高的方法是建立一个长度为 37 的数组 H , 令 $H[2^k \bmod 37] = k$ 。这里利用了一个小的数学技巧: $\forall k \in [0, 35], 2^k \bmod 37$ 互不相等, 且恰好取遍整数 1~36。修改之后的程序为:

```
int H[37];
for (int i = 0; i < 36; i++) H[(1 << i) % 37] = i;
while (cin >> n) {
    while (n > 0) {
        cout << H[(n & -n) % 37] << ' ';
        n -= n & -n;
    }
    cout << endl;
}
```

值得指出的是, lowbit 运算也是树状数组 (第 0x42 节) 中的一个基本运算。

GCC 编译器还提供了一些内置函数^①, 可以高效计算 lowbit 以及二进制数中 1 的个数。不过, 这些函数并非 C 语言标准, 有的函数更是与机器或编译器版本相关的。另外, 部分竞赛禁止使用下划线开头的库函数, 故这些内置函数尽量不要随便使用。

```
int __builtin_ctz(unsigned int x) .
int __builtin_ctzll(unsigned long long x)
返回  $x$  的二进制表示下最低位的 1 后边有多少个 0。
```

^① 关于 GNU C 编译器的部分内置函数的详细手册, 可参见配套光盘提供的参考资料。





基本算法

算法竞赛进阶指南



```
int __builtin_popcount(unsigned int x)  
int __builtin_popcountll(unsigned long long x)
```

返回 x 的二进制表示下有多少位为 1。



扫描全能王 创建